

Cover

AtlasBot v2 — MVP & Sprint Plan Coinbase Integration + Alpha Engine (Maximum ROI)

Prepared for: Luke • Date: October 13, 2025 Scope: UI/UX & DB prebuilt • This doc focuses on Coinbase wiring + Trading Algorithms + Research Note: Local-first. Uses Node (runtime) + Python (research) + ONNX (inference).

Objective: Ship a profitable, disciplined intraday crypto trading MVP with Coinbase integration, event-driven execution, and a small set of high-confidence edges governed by strict risk controls and meta-label gating.

Targets (rolling OOS): Sharpe ≥ 1.5 (7–30d), Max DD $\leq 10\%$, Positive expectancy ($R > 0$) after costs, slippage within budget, zero risk-control breaches.

ROI Lever Stack (What boosts profitability)

1) Signal Quality (small uncorrelated edges)

- Breakout + Volatility Gate (Donchian + ADX + ATR percentile).
- VWAP Mean Reversion ($|z|$ bands; stand down in high trend).
- Order Book Imbalance Scalper (top-of-book imbalance + delta flow spike; tight time stops).
- Meta-Label filter to accept only trades likely to hit $+1R$ before $-1R$.

2) Risk Sizing & Protection

- ATR-based position sizing by risk per trade (0.5–0.8% equity).
- Portfolio heat clamp ($\leq 3\%$). Correlation clamp for BTC/ETH.
- Time stops per strategy (scalps 10–25m; momentum 60–120m).
- Daily hard stop $-2R$; Kill-switch on spread/ATR bursts or API instability.

3) Execution Quality

- Post-only limits where time allows; IOC for micro-entry; TWAP for larger tickets.
- SlipGuard: abort entry when adverse mid move $> 0.5 \times \text{ATR}$ during slicing.
- Venue specifics: queue awareness, idempotency keys, retry with jitter; partial TP at $+1R$; ATR trail.

Architecture (Local-first, No External DB required for runtime)

Runtime: Node 20 + TypeScript for WS/REST + execution routing.

Research: Python 3.11 for backtests + meta-label training; export ONNX; runtime inference in Node.

State: flat files under ./var + (optional) your DB for UI/analytics already built.

Key Services (Processes)

- MarketFeed (WS): subscribes to Coinbase Market Data + User Order streams.
- Bar/Feature Engine: aggregates 1/5/15m bars; computes ATR/ADX/VWAP/Donchian/OBI/Delta/Spread%.
- Strategy Engine: computes signals; calls MetaFilter (ONNX).
- Risk Engine: sizes positions; checks heat/daily stop/kill-switch/time stops.
- Exec Router: post-only/IOC/TWAP; SlipGuard; fill handling; order/leg lifecycle.
- Journal/Logger: append-only JSONL logs; snapshots for UI; daily report.

Coinbase Integration Plan (Advanced Trade)

Channels & Endpoints

- WebSocket Market Data: ticker/L2 book for symbols (BTC-USD, ETH-USD, later top-20).
- WebSocket User Order Data: stream order acks/fills (or poll REST fallback).
- REST Orders v3: create/cancel/status; idempotency keys; maker/taker flags; rate-limit handling.

Best Practices

- Auth & time sync (UTC); HMAC per request; distinct API profiles for paper vs live.
- Idempotency: clientOrderId on parent & TWAP children; retry with backoff/jitter; safe cancel.
- Rate-limits: bucketed retry; circuit breaker to degrade to paper when limits hit.
- Validation: pre-flight size $\geq$ venue min; price increments adhere to tick size.
- Resilience: reconnect WS with sequence checks; heartbeat ping/pong; gap detection → REST recovery.

Data Contracts (Runtime ↔ DB)

Signals → DB.signals: {symbol, strategy, side, score, confidence, meta_prob, allowed, features?}.

Orders → DB.orders + order_legs; Fills → DB.fills; Positions rolled from fills; Risk events + Alerts.

Snapshots for UI: ./var/state/{positions.json, risk_state.json, status.json}; Blotter logs JSONL.

Strategy 1 — Breakout + Volatility Gate (Trend-Follow)

Setup

- Donchian high/low (N=20 on 5–15m). Confirm with $ADX \geq 18$, ATR in ≥ 60 th percentile (last M bars).

Entry/Exit

- Long: close > DonchianHi $\wedge$ gates pass. Short: mirror at DonchianLo.
- Stop: $1.3 \times ATR$ from entry; Partial TP: +1R (50%); Trail remainder with ATR or structure; Time stop 60–120m.

Notes

- Stand down in low-vol regimes to avoid chop; widen ladder in strong trends.
- Backtest sensitivity: N in [15–25], ADX in [15–22], ATR pctile in [0.55–0.75].

Strategy 2 — VWAP Mean Reversion (Intraday)

Setup

- Compute session VWAP and rolling σ ; z-score of price vs VWAP.

Entry/Exit

- Long when $\text{vwap_z} \leq -z_{\text{th}}$ and $\text{ADX} \leq 14$; spreads normal/contracting.
- Exit at VWAP touch or +1R; time stop 20–30m; avoid entry during high trend regimes.

Notes

- Filter with OBI/spread percentile to avoid catching falling knives; shrink size when spread widens.

Strategy 3 — Order Book Imbalance Scalper (Microstructure)

Setup

- Top-of-book or depth-5 imbalance (OBI) = $(\text{bid_sz} - \text{ask_sz}) / (\text{bid_sz} + \text{ask_sz})$; Delta flow spike confirms.

Entry/Exit

- Enter IOC small size only when spread $\leq$ median and delta aligns; exit on imbalance flip or 0.7–1.0R target; time stop $\leq$ 15m.

Notes

- High-frequency; must honor kill-switch and latency checks; disable during news/ATR bursts.

Meta-Label Filter (Accept/Reject Gate)

Label for each primary signal: 1 if +1R before −1R within T bars; else 0.

Features: regime (ADX, ATR pctile), spread pctile, OBI, delta, vwap_z, time-of-day, short-lag returns.

Model: LightGBM/XGBoost/logit; calibrate; export ONNX; runtime threshold $\theta \approx 0.62$ to target precision uplift.

Effect: fewer trades, higher average R; use walk-forward to select θ for best OOS Sharpe under DD cap.

Risk, Sizing, and Circuit Breakers

Per-trade risk: 0.5–0.8% equity; size = $(\text{equity} * \text{risk_bp}/1e4) / |\text{entry} - \text{stop}|$.

Portfolio heat cap: $\leq 3\%$ (sum of per-trade risk). Correlation clamp for BTC/ETH alignments.

Daily hard stop: $-2R$; pause new entries for session. Time stops per strategy; no averaging down.

Kill-switch: block entries when spread $> 2 \times$ median, 1m ATR $> 3 \times$ 3-day median, or API instability.

Execution Router & Fill Modeling

Order types: post-only limits (rebates), IOC for quick entries, TWAP for >\$5k.

SlipGuard: abort remainder when $|\text{mid_now} - \text{mid_before}| > 0.5 \times \text{ATR}$ during slicing.

Queue modeling: assume partial fills at touch; monitor realized vs. budgeted slippage; route adjustments.

Backtesting & Validation Protocol

Event-driven simulator with fees/spread/queue/slippage and timing rules identical to runtime.

Purged/embargoed K-fold to avoid leakage; Walk-forward: train 60d → test 7d; rotate windows.

Acceptance gates: Sharpe ≥ 1.5 OOS; Max DD $\leq 10\%$; hit-rate consistent (35–55%); tail losses trimmed (no trade $< -1.3R$).

MVP Sprint Plan — Overview (2-3 Weeks Fast-Track)

Sprint 0 (0.5-1d): Keys, Sandbox, Connectivity

- Set API keys/profiles; time sync; connect Market WS + User WS; REST ping; idempotency scaffolding.

Sprint 1 (1-2d): Feed + Features

- Aggregate 1/5/15m bars; compute ATR/ADX/VWAP/Donchian/OBI/Delta/Spread%; snapshot for UI; gap recovery.

Sprint 2 (1-1.5d): Orders & Router

- Create/cancel/status endpoints; post-only/IOC/TWAP; SlipGuard; order/leg lifecycle; logs + snapshots.

Sprint 3 (1-1.5d): Risk & Circuit Breakers

- Per-trade risk; heat; daily stop; time stops; kill-switch; correlation clamp; dry-run tests.

Sprint 4 (1-2d): Strategies + Paper

- Implement Breakout + VWAP MR; connect to risk/router; paper run with audits; tune thresholds.

Sprint 5 (0.5-1d): Live Micro-Size

- Flip to live at tiny size; verify fills; check safety rails; collect slippage stats.

Sprint 6 (2-3d): Backtests + Meta-Label

- Build Python sim; label; train; export ONNX; integrate gate; retune θ ; acceptance checks.

Sprint 0 — Detailed Tasks & DoD

Tasks

- Create API profiles (paper/live). Add clock skew alarm. Implement HMAC auth + signed headers.
- WS Market subscribe to BTC-USD, ETH-USD; parse ticker/L2. User WS subscribe; fall back to polling if needed.
- Set idempotency key generator; retry with jitter; HTTP error classifier; sandbox toggle.

DoD

- Market WS stable 30+ minutes with reconnects. REST create/cancel test returns success in sandbox. User updates observed or polling fallback verified.

Sprint 1 — Detailed Tasks & DoD

Tasks

- Bar aggregator (1/5/15m UTC) with boundary accuracy; gap detection and backfill; session VWAP.
- Feature engine with ATR/ADX/Donchian/OBI/Delta/Spread%; write snapshots and feature logs (optional).

DoD

- CSV/JSON snapshots update on schedule; features non-null; spread percentile reasonable; backfill after WS drop produces continuous series.

Sprint 2 — Detailed Tasks & DoD

Tasks

- Orders: create/cancel/status; order_legs for TWAP slices; IOC path; post-only flag; time-in-force.
- SlipGuard; partial TP at +1R; ATR trailing; trade & order logs; error localization (4xx vs 5xx).

DoD

- Paper fills with realistic slippage; legs visible; cancel works; audit trail written; no orphan orders after cancel.

Sprint 3 — Detailed Tasks & DoD

Tasks

- Risk manager: sizing (risk bp), heat tracking, daily stop, correlation clamp, time stops per strategy.
- Kill-switch triggers on spread/ATR/API errors; safe overrides (time-limited); status snapshot for UI.

DoD

- Forced $-2R$ test halts entries; kill-switch fires on synthetic spread spike; resume only after manual override or metrics normalize.

Sprint 4 — Detailed Tasks & DoD

Tasks

- Implement Breakout+Vol and VWAP MR with config thresholds; integrate with risk + router.
- Paper run during active hours; collect trades; compute preliminary expectancy; tweak gates.

DoD

- Paper equity curve positive vs. baseline; parameters recorded; no risk breaches; SlipGuard events reduce adverse fills.

Sprint 5 — Detailed Tasks & DoD

Tasks

- Enable live with tiny notional; watch slippage, maker/taker mix; ensure post-only behaves as expected.
- Verify Close All and Pause/Resume; emergency stop runbook executed once.

DoD

- Live fills present; blotter and positions consistent; no stray working orders; daily stop respected.

Sprint 6 — Detailed Tasks & DoD

Tasks

- Backtest engine parity with runtime rules; generate labels; train meta; calibrate; export ONNX; integrate gate; choose θ .
- Walk-forward analysis; parameter selection locked; produce report and daily MD summary.

DoD

- OOS Sharpe ≥ 1.5 and Max DD $\leq 10\%$ on last 30d; meta-filter improves expectancy vs. raw; ONNX loads and gates live entries.

Acceptance Tests (System)

- Daily stop breach prevents new entries; orders not sent while paused; overrides expire on time.
- Kill-switch blocks entries during spread 99th pctile or $1m\ ATR > 3 \times \text{baseline}$; resumes after normalization.
- SlipGuard cancels remainder when mid moves adverse $> 0.5 \times ATR$.
- Close All exits positions and cancels working orders deterministically; no dangling state.

Risk Playbook & Operations

Runbook — Normal Ops

- Start in paper; confirm data fresh; enable live; monitor TUI/UI metrics; end-of-day report & journal.

Runbook — Emergencies

- Immediate Pause → Close All → Cancel Working → check WS/REST health; if API unstable, switch to paper.
- Rotate keys if auth anomalies; review logs for error spike root-cause; document incident in journal.

Parameter Baselines & Budgets

Risk: 70 bp per trade; heat cap 300 bp; daily stop $-2R$.

Breakout: $N=20$; $ADX \geq 18$; $ATR \text{ pctile} \geq 0.60$; Stop $1.3 \times ATR$; TP ladder $1R/2R$; time stop 60–120m.

VWAP MR: $|z| \geq 1.2$; $ADX \leq 14$; exit to VWAP; time stop 25m.

OBI Scalper (post-MVP): $OBI \geq 0.6$ with delta spike; time stop $\leq 15m$; target $0.7-1.0R$.

Meta $\theta \approx 0.62$ (calibrate). Slippage budget: $\leq$ planned model; maker ratio target $\geq 40\%$ when feasible.

Appendix — Pseudocode (Signal → Gate → Size → Execute)

Core Loop (per new 5m bar)

- features = computeFeatures(symbol)
- for s in strategies: out = s.compute(features); if !out: continue
- ok, p = meta.allow(vectorize(features, s)); if meta.enabled and !ok: continue
- qty = risk.size(equity, price, out.stop, risk_bp); if qty==0: continue
- if risk.allowed(symbol, qty): router.place(parent(limit|ioc|twap), slipguard)

SlipGuard

- abort if $|\text{mid_now} - \text{mid_before}| > 0.5 \cdot \text{ATR}$ while slicing

Appendix — Coinbase Endpoints & Error Handling

REST: POST /orders (create), GET /orders/{id}, DELETE /orders/{id}, GET /orders (list), GET /fills.

WS Market Data: subscribe to products, channels=ticker|l2; WS User Data: orders/fills stream.

Errors: classify 4xx (validation/rate-limit) vs 5xx (retryable). Retry policy with jitter; circuit breaker to paper.

Final Notes & Disclaimers

No strategy guarantees profits. This MVP emphasizes risk-first engineering, robust execution, and evidence-backed edges to maximize probability of positive risk-adjusted returns.

Next steps after MVP: add OBI scalper, dynamic universe selection, venue routing, and adaptive slippage models.